

좋은 하네스는 어떻게 만드는가

하네스 진입장벽 낮추기

0. 발표 목적

차근차근 하네스와 친해지자

1. 하네스를 제작하는 전체적인 프로세스를 따라가며 진입장벽을 낮추기
2. 하네스를 만들면서 느낀 인사이트와 꿀팁 공유

1. 하네스 개발을 시작하는 전체적인 목표

작고 명확한 단위의 Skill부터 시작하자.

처음 목표는 **작고 명확한 단위의 Skill**과 Eval Set

- LLM 출력이 **반복가능**하고 **재현가능**하고 **측정 가능**하도록 만들기
- 그리고 내가 만든 스킬이 올바르게 작동하는지를 어떻게 검증했는지, 그리고 어떤 성공 기준을 설정하고 개선했는지와 같은 엔지니어링 과정을 설명할 수 있어야 한다.

2. 스킬 개발의 프로세스

범용적인 주제보다는, 내 문제 상황에 맞는 작은 스킬을 먼저 만들어보자!

최소한 이 스킬만큼은 누구보다 잘만들었다 할 정도로 작은 스코프의 스킬을 갖고 닦기

1. 문제 스코프를 작게 줄인다.
2. 하나의 Skill을 정한다.
3. 그 Skill이 잘 작동했다는 것을 확인할 수 있는 성공 기준을 만든다.
4. 작은 Eval Set을 구성한다.
5. 여러 번 실행해보고 결과를 기록한다.
6. V1, V2, V3처럼 반복적으로 개선한다. (사이클)

3. 좋은 스킬을 만들기 위한 기준

기준을 명확히 하고 준비하기

- 어떤 상황에서 이 Skill을 사용할지 명확히 한다.
- 이 Skill이 해결해야 하는 문제를 좁힌다.
- 성공 기준을 하나라도 분명히 둔다.
- 실패 사례를 기록한다.
- 반복되는 실수는 규칙이나 테스트로 만든다.
- 개선 전후의 차이를 기록한다.

4. 성공 기준과 평가 기준의 중요성

기준을 명확히 하고 준비하기

AI 엔지니어링에서 가장 중요한 역량 중 하나는, **명확한 성공 기준을 세우는 능력**

성공 기준과 평가기준을 명확히 세우자.

(첨언) 성공기준과 평가 기준을 세울 수 있을 때 고려할 수 있는 질문

- 이 작업이 성공했다는 것을 무엇으로 알 수 있는가?
- 이전 버전보다 개선되었다는 것을 어떻게 확인할 수 있는가?
- 결과를 사람이 봐야 하는가, 아니면 Boolean처럼 통과/실패로 판단할 수 있는가?
- 정량 지표로 볼 수 있는 것은 무엇인가?
- 정성 판단이 필요한 부분은 어떻게 샘플링하고 기록할 것인가?

5. Eval Set와 벤치마크 구성

기준을 명확히 하고 준비하기

Eval Set은 단순히 테스트 데이터를 모아두는 것이 아니라,
내 Skill이 어떤 조건에서 잘 작동하고 어떤 조건에서 실패
하는지 확인하기 위한 기준 세트이다.

추천: 엔트로픽의 evals와 openai의 dataset 공식 문서

6. 좋은 eval을 만드는 방식

기준을 명확히 하고 준비하기

Eval을 만들 때는 가능한 한 테스트 결과가 객관적으로 나오도록 설정하자.

예시: AI말투 제거 스킬 `im-not-ai`의 eval 기준

- 고유명사, 수치, 인용문이 보존됐는가
- 변경률이 허용 범위 안인가
- 특정 AI 패턴이 감소했는가
- 장르와 말투가 유지됐는가

7. 반복 실행과 신뢰구간

LLM은 비결정적이다. 반복해서 평가해야 한다.

통계나 데이터 시뮬레이션에서 불확실한 현상을 다룰 때처럼,
LLM 평가에서도 어느 정도의 반복 실행과 신뢰구간 개념이 필요하다.

여러 번의 반복 실행 경험을 통해,
최소 몇 번은 실행해야 내가 이 결과를 어느 정도 신뢰할 수 있는지에 대한 감각을 기르는 것

8. 테스트 비용과 작업 단위 쪼개기

LLM을 통한 테스트는 토큰에서 병목이 있다.

많이 반복할 수록 좋은 것이 아닌가?

- LLM 기반 테스트는 기본적으로 토큰 소모 비용이 발생한다.
- 현실적으로 토큰 비용이 발생하기에 효율적인 횟수를 찾는 것이 중요하다.
- 테스트 구조를 수정하거나, 테스트 단위를 작게 유지하여 컨텍스트를 줄이자.
- 코드로 검증 가능한 부분은 따로 분리하자!

10. 코드로 강제할 것과 LLM에게 시킬 것

LLM은 비결정적이다.

프로그램으로 처리할 것과 LLM으로 처리할 것의 경계를 명확히 나눠야한다.

- 이 또한 AI엔지니어의 역량이다.
- 최대한 프로그램이나 코드로 강제할 수 있는 부분을 찾고 처리하는 것이 중요하다.
- LLM 모델이 판단하고 처리하는 것보다, 코드나 프로그램을 실행하는 것이 싸고 빠르고 안정적이다.

예를 들어 다음과 같은 부분은 프로그램이나 테스트로 강제할 수 있다.

- 빌드가 되는지
- 테스트 코드가 변경되지 않았는지
- 특정 파일만 수정했는지
- 금지된 언어나 프레임워크를 사용하지 않았는지
- 정해진 출력 형식을 지켰는지
- 필수 메타데이터가 포함되었는지
- 특정 hook이나 command가 실행되었는지

부록1. 토큰 사용과 비용 최적화

토큰 최적화 물론 중요하다.

초보자 단계에서는 토큰을 너무 아끼려고 하기보다, 일단 많이 써보자
- 많이 써봐야 어디까지 가능한지, 어떤 작업에서 비용이 많이 드는지,
어떤 구조가 비효율적인지 감을 잡을 수 있다.

물론 실무적으로 반복 작업이 많아지면 토큰 최적화가 필요하다.

부록2. 통계와 시뮬레이션 학습의 필요성

이제 불확실성을 올바르게 수치화하는 능력이 중요하다.

LLM의 품질을 측정하려면 앞으로 통계와 시뮬레이션 감각이 점점 더 중요해질 수 있다.

- 불확실한 결과를 반복 실행하고, 신뢰구간으로 이해하는 능력이 필요해짐.
- 일상적인 문제(예: 몇시에 집에서 나오면, 제시간안에 출근할수 있을까?)에서 시작해서 어느 정도의 신뢰범위를 지정하는 것이 유의미할지를 경험해보는 것도 추천
- 이를 LLM 평가에 접목하면, Skill은 몇 번 실행해야 신뢰할 수 있을까? 신뢰가능한 구간은 어느 정도일까?

부록3. 공식문서를 잘 살펴보자

올바른 프롬프팅 엔지니어링을 하는 가이드가 공식문서에 친절히 작성되어 있다.

공식 문서를 적극적으로 보자.

- 공식 문서에는 **LLM 기반 검색이나 채팅 기능**이 붙어 있는 경우도 많기 때문에, 필요한 부분을 질문하면서 찾을 수 있다.
- **공식 문서는 최신 모델을 가장 잘 이해하는 사람들이 쓴 기준**이기 때문에, 감으로 개선하는 것보다 더 좋은 근거
- 공식 문서의 기준을 참고해 이런 방식으로 Eval을 구성했고, 반복 실행 결과 이런 개선이 있었다로 주장.

결론, 지금 시작한다면?

작은 스킬 제작부터 시작해보자.

동작 가능한 가장 작은 단위부터 시작하자.

- 가장 작은 변환 작업 하나를 고른다.
- 그 작업을 수행하는 Skill V1을 직접 작성한다.
- 이 Skill이 잘 작동했는지 판단할 Eval Set을 만든다.
- 최소 5회에서 10회 정도 실행해본다.
- 실패 사례를 기록한다.
- 반복되는 실패를 테스트나 규칙으로 만든다.
- Skill V2로 개선한다.
- 개선 전후를 비교한다.
- 필요하면 30회 이상 반복 실행해 신뢰도를 본다.
- 이 과정을 문서화한다.



결론, 중요 내용 요약

첫째, 처음부터 거대한 하네스를 만들려고 하지 않는다. 작고 명확한 Skill 하나를 제대로 설계하고 평가하는 경험이 먼저다.

둘째, AI 엔지니어링의 핵심은 성공 기준이다. 잘됐는지 판단할 수 없다면 개선할 수도 없다.

셋째, LLM의 결과는 반복 실행과 Eval Set을 통해 봐야 한다. 한 번 잘 나온 결과는 운일 수 있다.

넷째, 프로그램으로 강제할 것과 LLM에게 맡길 것을 나눠야 한다.

좋은 하네스는 LLM을 풀어놓는 구조가 아니라, 가드레일 안에서 자유롭게 탐색하게 하는 구조다.

다섯째, 공식 문서를 적극적으로 참고해야 한다.

감으로 개선하기보다 공식 문서의 기준을 바탕으로 실험하고 기록하는 것이 훨씬 강한 포트폴리오가 된다.

여섯째, 토큰 최적화는 중요하지만 초반에는 많이 써보며 감각을 익히는 것이 우선이다.

나중에 반복 작업이 많아지면 어떤 부분에서 비용이 급격히 늘어나는지 분석하면 된다.

마지막으로, “AI가 잘했다”가 아니라 “내가 어떤 기준과 구조를 설계했기 때문에 더 나은 결과가 나왔다”고 설명할 수 있어야 한다.

그 경험을 작게라도 빨리 만드는 것이 좋은 AI 엔지니어링 학습의 출발점이다.

